

ui.pagination 全面详解

ui.pagination 是 NiceGUI 基于 Quasar 的 QPagination 组件封装的分页元素，用于实现多页内容的导航控制，支持页码范围配置、首尾页快捷链接、页码切换回调等核心功能，广泛适用于表格数据分页、列表内容分页、搜索结果分页等场景。其核心特性包括灵活的页码配置、状态绑定、禁用控制等，可快速实现交互友好的分页导航效果。

一、核心概念与基础特性

1. 本质与用途

- 本质：封装 Quasar 的 QPagination 组件，通过配置页码范围（最小页、最大页）和初始页码，生成可视化的分页导航控件，支持用户点击页码或快捷链接切换页面。
- 核心用途：在大量数据或内容展示场景中，将内容拆分到多个页面，减少单次加载压力，提升用户浏览体验，常见于数据表格、文章列表、商品列表等场景。
- 关键机制：支持手动点击切换页码，也可通过程序控制页码变更；页码切换时触发回调函数，便于联动更新页面内容；支持显示首尾页快捷链接，提升导航效率。

2. 基础结构

ui.pagination 为独立控件，无需嵌套子元素，通过初始化参数配置核心功能，配合回调函数或数据绑定实现页码与内容的联动，示例结构如下：

```
from nicegui import ui

# 初始化分页控件：页码范围1-10，显示首尾页链接，初始页码为3
def on_page_change(page):
    # 页码切换时更新内容（此处示例为更新文本提示）
    page_label.set_text(f'当前页码: {page}')

pagination = ui.pagination(
    min=1,
    max=10,
    direction_links=True,
    value=3,
    on_change=on_page_change
)

# 显示当前页码
page_label = ui.label(f'当前页码: {pagination.value}').classes('mt-2')

ui.run()
```

二、初始化配置项

初始化 `ui.pagination()` 时可通过参数配置页码范围、导航样式和回调函数，参数说明如下：

参数名	类型	说明
min	int	最小页码（必填，定义分页的起始页码，通常为 1）
max	int	最大页码（必填，定义分页的结束页码，需大于等于 min）
direction_links	bool	是否显示首尾页快捷链接（默认未明确指定，需手动设置为 True 启用）
value	int	初始页码（默认值为 min，即默认选中最小页码）
on_change	Callable	页码切换时触发的回调函数（参数为当前选中的页码值）

配置示例

```
# 页码范围2-8，不显示首尾页链接，初始页码为5，切换时弹窗提示
def handle_change(page):
    ui.notify(f'已切换到第 {page} 页')

ui.pagination(
    min=2,
    max=8,
    direction_links=False,
    value=5,
    on_change=handle_change
)
```

三、核心属性

ui.pagination 继承 NiceGUI 基础元素的通用属性，支持样式配置、状态绑定等，关键属性如下（含可设置属性）：

属性名	类型	说明
classes	str	元素的 CSS 类名（支持 Tailwind、Quasar 类，如 justify-center 水平居中）
props	str	Quasar 组件属性（用于扩展功能，如 color=primary 设置颜色、size=lg 调整大小）
style	str	内联 CSS 样式（如 margin-top: 20px；调整上边距）
value	BindableProperty	当前选中的页码（支持双向绑定，可通过变量同步页码状态）
visible	BindableProperty	元素可见性（布尔值，支持动态绑定）
enabled	BindableProperty	元素启用状态（布尔值，True 为启用，False 为禁用，禁用后无法切换页码）
min	int（可设置）	最小页码（初始化后可通过 pagination.min = 新值 动态修改）
max	int（可设置）	最大页码（初始化后可通过 pagination.max = 新值 动态修改）

属性名	类型	说明
direction_links	bool (可设置)	是否显示首尾页链接（初始化后可通过 <code>pagination.direction_links = True</code> 启用）
html_id	str	HTML DOM 中的元素 ID（版本 2.16.0 新增，用于精准定位）
is_deleted	bool	元素是否已被删除（只读属性）
parent_slot	Slot None	父容器的插槽（可手动设置元素所属父插槽）

属性使用示例

```
# 居中显示、蓝色样式、大号尺寸的分页控件
pagination = ui.pagination(
    min=1,
    max=15,
    direction_links=True,
    value=1
).props('color=blue size=lg').classes('justify-center mt-6')

# 动态修改属性
def update_pagination():
    pagination.min = 3 # 修改最小页码为3
    pagination.max = 20 # 修改最大页码为20
    pagination.direction_links = False # 隐藏首尾页链接
    pagination.value = 5 # 切换到第5页

ui.button('更新分页配置', on_click=update_pagination).classes('mt-4')
```

四、核心方法

ui.pagination 提供丰富的方法用于控制页码、绑定状态、管理元素等，常用方法分类如下：

1. 页码与状态控制方法

方法名	作用	示例
set_value(value)	设置当前选中页码（value 需在 min 和 max 之间）	<code>pagination.set_value(4)</code>
enable()	启用分页控件（启用后可切换页码）	<code>pagination.enable()</code>
disable()	禁用分页控件（禁用后无法点击切换）	<code>pagination.disable()</code>
set_enabled(value: bool)	设置启用状态（True 启用，False 禁用）	<code>pagination.set_enabled(False)</code>
set_visibility(visible: bool)	设置可见性（True 显示，False 隐藏）	<code>pagination.set_visibility(False)</code>

2. 绑定方法

支持将页码、启用状态、可见性与外部变量绑定，支持单向 / 双向绑定，核心方法如下：

方法名	作用	关键参数
<code>bind_value(target_object, target_name='value')</code>	双向绑定页码到目标对象的属性	<code>target_object</code> : 绑定目标对象; <code>target_name</code> : 绑定的属性名 (默认 'value')
<code>bind_value_from(...)</code>	单向绑定 (从目标对象同步到分页控件)	同 <code>bind_value</code> , 仅单向同步 (目标对象属性变化触发页码更新)
<code>bind_value_to(...)</code>	单向绑定 (从分页控件同步到目标对象)	同 <code>bind_value</code> , 仅单向同步 (页码变化触发目标对象属性更新)
<code>bind_enabled(...)</code>	双向绑定启用状态到目标对象的属性	同 <code>bind_value</code> , 绑定的属性为启用状态 (默认 'enabled')
<code>bind_enabled_from(...)</code>	单向绑定启用状态 (从目标对象同步)	同 <code>bind_enabled</code> , 仅单向同步
<code>bind_enabled_to(...)</code>	单向绑定启用状态 (从分页控件同步)	同 <code>bind_enabled</code> , 仅单向同步
<code>bind_visibility(...)</code>	双向绑定可见性到目标对象的属性	<code>value</code> : 可选, 指定目标值匹配时才显示分页控件

3. 其他常用方法

方法名	作用	示例
<code>tooltip(text)</code>	为分页控件添加鼠标悬浮提示	<code>pagination.tooltip('页码导航')</code>
<code>update()</code>	强制更新客户端的分页控件状态 (如动态修改属性后刷新)	<code>pagination.update()</code>
<code>add_resource(path)</code>	为分页控件添加资源文件 (如自定义 CSS、JS)	<code>pagination.add_resource('./static')</code>
<code>ancestors(include_self=False)</code>	迭代获取所有祖先元素	遍历祖先元素: <code>for elem in pagination.ancestors(): print(elem)</code>
<code>mark(*markers)</code>	为元素添加标记 (用于测试或元素查询)	<code>pagination.mark('data-pagination', '2024')</code>
<code>delete()</code>	删除分页控件元素	<code>pagination.delete()</code>

方法使用示例

```
from nicegui import ui

# 绑定页码到外部变量
class PageState:
    current_page = 1

state = PageState()

# 双向绑定: 分页控件页码与 state.current_page 同步
pagination = ui.pagination(
    min=1,
    max=10,
    direction_links=True
).bind_value(state, 'current_page')

# 显示绑定的页码值
ui.label('绑定的页码: ').bind_text_from(state, 'current_page', lambda p: f'第 {p} 页')

# 程序控制页码
ui.button('跳转到第3页', on_click=lambda: setattr(state, 'current_page', 3)).classes('mt-4')
ui.button('禁用分页', on_click=pagination.disable).classes('m1-2')
ui.button('启用分页', on_click=pagination.enable).classes('m1-2')

ui.run()
```

五、事件处理

1. 核心事件：页码切换事件

通过初始化参数 `on_change` 或方法 `on_value_change` 绑定页码切换回调，两种方式效果一致：

```
# 方式1: 初始化时绑定
def on_change(page):
    ui.notify(f'页码切换: {page}')
pagination = ui.pagination(min=1, max=5, on_change=on_change)

# 方式2: 通过方法绑定
def handle_value_change(e):
    ui.notify(f'当前页码: {e.value}')
pagination.on_value_change(handle_value_change)
```

2. 通用事件绑定

通过 `on()` 方法绑定 DOM 事件（如点击、鼠标悬浮），支持 Python 或 JavaScript 回调：

```
# 分页控件点击事件 (Python 回调)
pagination.on('click', lambda e: ui.notify('点击了分页控件'))

# 鼠标悬浮事件 (JavaScript 回调)
pagination.on('mouseover', js_handler='(e) => console.log("鼠标悬浮分页控件")')
```

六、高级用法

1. 样式深度自定义

结合 `props`、`classes` 和 `style` 实现个性化样式，示例：

```
# 自定义颜色、尺寸、间距的分页控件
ui.pagination(
    min=1,
    max=8,
    direction_links=True,
    value=2
).props(
    'color=teal', # 颜色为青绿色
    'size=md',    # 中等尺寸
    'rounded'     # 圆角样式
).classes('justify-center mt-8').style('font-weight: 600;')
```

2. 动态修改页码范围

根据业务逻辑（如数据加载后更新总页数）动态修改 `min` 和 `max` 属性，示例：

```
from nicegui import ui

pagination = ui.pagination(min=1, max=5, direction_links=True)

# 模拟加载更多数据后，更新最大页码为15
def load_more_data():
    pagination.max = 15
    pagination.update() # 刷新分页控件
    ui.notify('数据加载完成，总页数更新为15')

ui.button('加载更多数据', on_click=load_more_data).classes('mt-4')
```

3. 分页与内容联动（实战场景）

结合列表或表格，实现页码切换时更新展示内容，示例：

```
from nicegui import ui

# 模拟数据：1-30条数据
data = [f'内容条目 {i}' for i in range(1, 31)]
items_per_page = 5 # 每页显示5条
current_items = ui.label('').classes('text-lg')
```

```
def update_content(page):
    # 计算当前页显示的数据范围
    start = (page - 1) * items_per_page
    end = start + items_per_page
    page_data = data[start:end]
    # 更新显示内容
    current_items.set_text('\n'.join(page_data))

# 初始化分页控件，总页数=总数据量/每页条数（向上取整）
total_pages = (len(data) + items_per_page - 1) // items_per_page
pagination = ui.pagination(
    min=1,
    max=total_pages,
    direction_links=True,
    on_change=update_content
)

# 初始显示第一页内容
update_content(1)
ui.element('div').bind_content_from(current_items, 'text').classes('mt-4 whitespace-pre-line')

ui.run()
```

七、注意事项

1. 页码范围约束： `max` 必须大于等于 `min`，否则分页控件可能无法正常显示；动态修改 `min` 或 `max` 后，需确保当前 `value` 在新的 `[min, max]` 范围内，否则可能出现选中页码异常。
2. 初始页码默认值：若未指定 `value`，则默认选中 `min`（最小页码），无需额外配置。
3. 方向链接显示： `direction_links` 默认为未启用状态，需手动设置为 `True` 才会显示首尾页快捷链接（通常为「<<」和「>>」图标）。
4. 版本兼容性： `html_id` 属性需 NiceGUI 2.16.0+ 版本支持， `bind_enabled` 的 `strict` 参数需 3.0.0+ 版本支持，使用时需确认版本匹配。
5. 禁用状态影响：禁用分页控件（ `disable()` 或 `set_enabled(False)` ）后，用户无法点击切换页码，但仍可通过 `set_value()` 方法程序控制页码。

通过以上配置与方法， `ui.pagination` 可灵活满足从简单页码导航到复杂数据分页的各类需求，是 NiceGUI 中实现多页内容管理的核心组件之一。